KRPCLAB · BOOSTERLAND

How the booster lander works

Bringing a spent first stage back to the KSC launch pad, Super Heavy style, from a kRPC script

The booster separates at about 20-35 km, still moving downrange at several hundred metres per second. The autopilot flips it, burns back toward the pad, falls through the atmosphere steering with its body and its airbrakes, and lights the engines once more for a suicide burn that puts the legs on the pad.

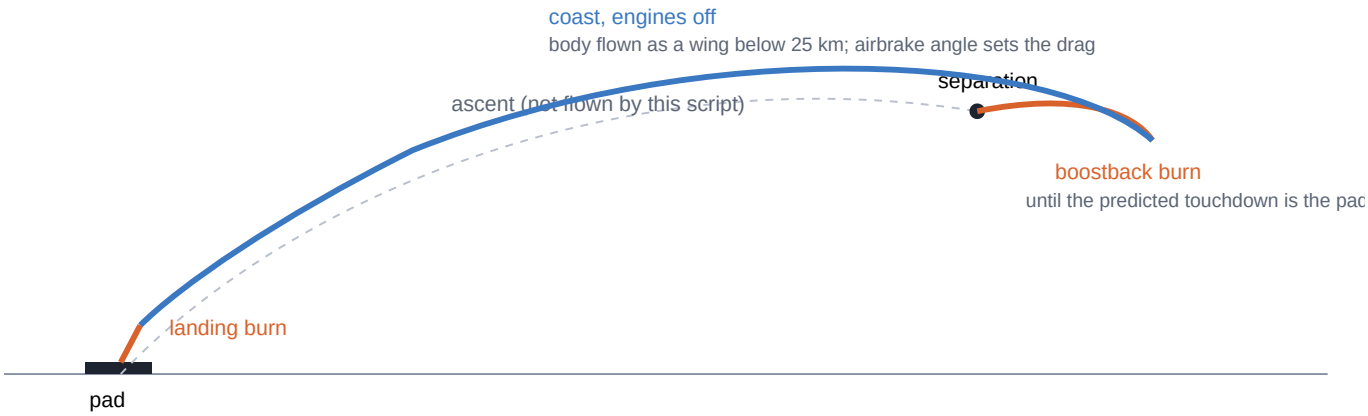


Figure 1. The flight profile, not to scale. Orange is engine-on, blue unpowered. Every part of it is re-decided each tick from a fresh prediction of where the booster will touch down.

The one idea: re-predict the landing every tick

There is no stored plan. About twenty times a second the autopilot reads the booster's position and velocity and integrates the rest of the flight to the ground in local Python: gravity, drag, body lift, the planet's rotation and the landing burn itself. Every decision that tick (where to point, how much throttle, when boostback ends, when the landing burn lights, how far the airbrakes open) is taken from that fresh prediction, so a disturbance is corrected without anything having to detect it as one.

`autoland.py` is the phase machine and the only code that talks to the game. `trajectory.py` is the propagator, `guidance.py` holds the control laws as pure functions, and `environment.py` keeps the world data the propagator needs (density, speed of sound, a drag curve, a lift slope), cached so the integration makes no remote calls.

- **It flies the landing burn too.** The prediction coasts to where the suicide burn will start and keeps integrating with the engines on, through the same throttle law the vehicle flies (`trajectory.landing_command` is shared by both).
- **RK4.** Drag goes as v² through a density that changes by a factor of e every 5 km; a first-order step would bias the touchdown.
- **Drag is a curve against Mach,** probed with kRPC's `simulate_aerodynamic_force_at`, which simulates the vessel *as it is now*: deploy angle included. The whole curve is re-swept every 10 s once the booster falls nose-first, and immediately after the airbrakes move.
- **Lift is discounted by how well the booster holds its angle.** The side-force slope is scaled by the square of the measured aoa tracking (held ÷ commanded, steady samples only), because lift goes as the angle squared

and a weathervaning booster holds a fraction of what it is told.

- **Heights are measured from the feet,** terrain under the legs clamped against the pad's radius; the rotation vector is measured from the vessel's velocity in two frames.

The flight, phase by phase

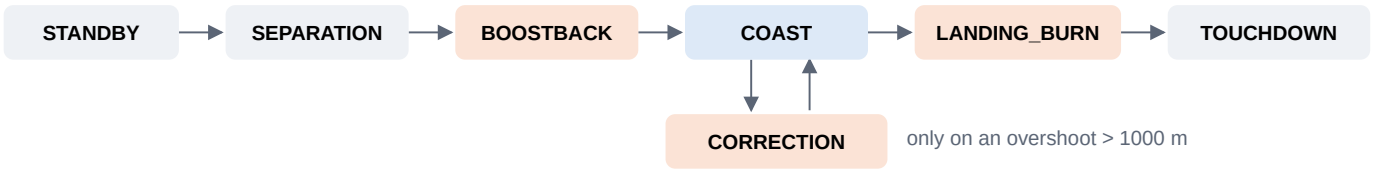


Figure 2. The phase machine. Orange phases burn the engines. CORRECTION is a short relight that COAST can enter and return from.

STANDBY waits for START	On START: RCS permitted, every airbrake deployed at 60°, the middle of the range the deploy law uses (pitch/yaw/roll axes left as the craft has them), every liquid engine activated and grouped for allocation, the autopilot engaged on the current attitude.
SEPARATION a few seconds	`proximity.py` wraps each nearby craft in a sphere and marches the booster's motion forward against each, coasting and at full thrust along the boostback aim. The flip starts on the first tick nothing is in the way; only getting closer counts as a conflict; a 5 s backstop flips anyway.
BOOSTBACK flip and burn	Aim opposite the predicted horizontal miss and go to full throttle before the turn is finished, so the gimbal does the turning; big turns are routed over the top. Throttle tapers with the remaining Δv; the burn ends when the predicted touchdown, landing burn included, has stayed within 190 m of the pad for four ticks. Roll puts the fins in the trajectory plane.
COAST engines off	Fall back toward the pad. Below 25 km the body is flown as a wing, and for the whole coast the airbrakes' deploy angle is stepped against the along-track miss (next section). At the predicted ignition point → LANDING_BURN. On a predicted *overshoot* over 1000 m with height and time left → CORRECTION.
CORRECTION relight, rare	A throttled-down (≤ 40%) relight, steered within 50° of retrograde, which in the game can only shorten the flight. So it is entered only when the miss is long (or mostly cross-track), never on a short miss, and only above 1000 m: a disturbance the air cannot steer out. Each burn is gated on a propagated gradient (metres closed per m/s), ends under 15 m, and is cut if the miss grows past 1.3 × (or 60 m beyond) the best it has reached. At most three burns, 20 s apart. Stuck off-aim and not turning, it lights at 25% so the gimbal can turn it.
LANDING_BURN suicide burn	Gear down on the first tick. Throttle follows a constant-deceleration descent profile (next-but-one section). The burn steers with ZEM/ZEV terminal guidance on the propagator's predicted touchdown; below 250 m it only nulls sideways speed, its attitude reference being retrograde held inside a 15° cone round vertical (vertical when ascending).
TOUCHDOWN hand back	Engines cut on reported contact (or a hover a few metres up). Every exit path runs the same shutdown: throttle zero, autopilot off, SAS on, panel removed.

Two controls in the coast: lift and drag

Engine burns after boostback can only shorten the flight. The air can push both ways, and the coast uses it twice: the angle of attack makes a side force, and the airbrakes' deploy angle sets the drag.

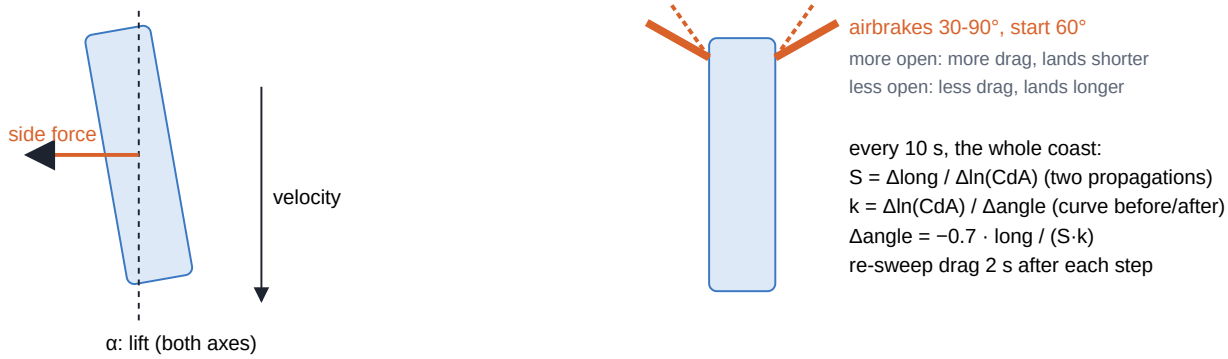


Figure 3. Left: tilting a few degrees off the velocity turns the airflow into a side force, roughly as α·|α|. Right: the deploy angle is a range control in both directions; the drag probe simulates the new shape, so the next prediction already flies it.

Lift (below 25 km). The angle is solved each tick: propagate small test angles in-plane and out-of-plane, invert the 2×2 to null downrange and cross-track, then propagate the answer and keep it only if it lands nearer than doing nothing (else halve once, else hold retrograde). The commanded angle is capped at 10°. The solved trajectory is the stored prediction, so the landing-burn trigger and the drag probe see the path the booster is about to fly.

Drag (the whole coast). `Autoland.run_deploy` takes a Newton step every 10 s on a gain it works out for itself, so no constant belongs to one craft. The gain is the product of two measured quantities. *S*, metres of along-track miss per unit of ln(Cd·A), comes from flying the propagator twice, the second time with the drag curve scaled by 1.05 and the same steering; it is exact for this trajectory and recomputed every step. *k*, ln(Cd·A) per degree of deploy, comes from the drag curve swept 2 s after a step (once the brakes have finished moving) against the curve before it, median over the Mach bins; the first step is a fixed 10° that takes the first reading. The step is −0.7·long/(S·k), rate-limited to 20°, within 30-90°. `long` is measured along the horizontal velocity, the axis drag acts on. Starting at 60° leaves the law room both ways: boostback aimed with that drag, so it can add drag as well as shed it.

> **WHY THE COAST GUARDS EXIST**
> The wing waits for 25 km because above it the drag curve is still being re-measured nose-first. Its answer is re-propagated before it is flown, because near the ground the sensitivity collapses and a raw solve would return the maximum angle. The deploy law steps only after the drag curve has seen its last step, because until then the prediction does not contain it.

The landing burn: a descent profile

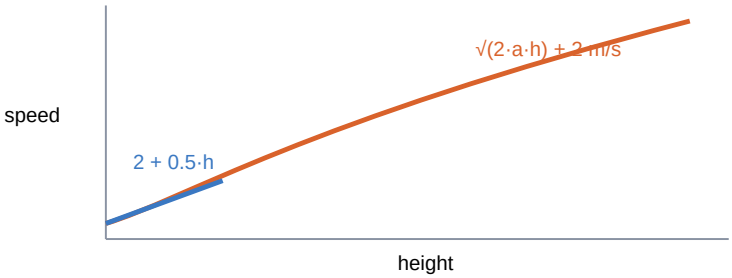
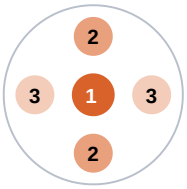


Figure 4. The throttle tracks the lower of the two curves: a constant-deceleration stop at 95% of available thrust, blending into a constant-rate final descent. Feed-forward plus a trim of 6 m/s² per m/s of error, divided by the booster's own maximum acceleration, so a 9 g and a 3.7 g booster get the same loop.

The tracked speed fades sideways speed out over the last 250 m. Thrust along the vertical cannot remove sideways speed, so on total speed a sliding booster would read as too fast, open to the cap (which leaves the divert no tilt to spend), climb, be cut by the go-around guard, fall, and repeat. On the descent rate the throttle sits between 0 and the cap and the velocity-nulling divert keeps its authority to the end. Below 60 m an ascending booster gets zero throttle; below 12 m at under 8 m/s the throttle is capped at hover plus 0.12.

Steering: ZEM/ZEV guidance, gains 6 and 4 (the energy-optimal double integrator), aimed at the propagator's predicted touchdown. The tilt is capped at 15° and at `acos(throttle ÷ cap)`, the angle the spare throttle can pay for; the throttle is opened by 1/cos(tilt). Below 250 m the position term is dropped and only the sideways velocity is nulled, down to 4 m.

Engine allocation: center first, then pairs



demand 0.1 → engine 1 at 50%, the rest at 0

demand 0.4 → engine 1 at 100%, pair 2 at 50%

demand 1.0 → everything at 100%

groups: an engine on the thrust axis; two opposite each other; an odd ring as one group.
each group's thrust passes through the axis, so lighting it adds no torque.

Figure 5. The throttle guidance asks for is a fraction of the cluster's thrust. It is filled group by group, each to full before the next lights, through per-engine thrust limits with the main throttle at 1.

At START, `EngineCluster` reads each liquid engine's position in the vessel frame and its thrust, and `engines.group_engines` groups them about the thrust-weighted centroid: within 0.3 m of the axis is the center group; the rest form rings, and in a ring an engine is paired with the one whose offset most nearly cancels its own; whatever cannot be paired lights together. Every phase's throttle goes through `set_throttle`, which writes only the limits that moved. Guidance's maximum acceleration reads `max_thrust`, which ignores limits, so the planner always sees the whole cluster. A single-engine booster is one group and flies as before. At hand-back every limit returns to 100%.

Smaller rules

- **RCS:** in COAST the valve opens only when the attitude controller has been saturated (input ≥ 0.95 for 0.5 s) and is not closing; elsewhere it is a relay on pointing error.
- **Roll is held** with the roof in the trajectory plane, seeded from the zenith once and carried forward.
- **Big turns go over the top,** so the engines never swing down into the airstream.
- **Every log ends with the loop rate** per phase (game-seconds between commands, wall-seconds of work), and the config line carries a fingerprint of the defaults.

- **Throttle divergence is logged:** a commanded throttle the vessel does not report is a physical throttle axis winning.

How it is measured

Each flight writes a numbered log (`logs/LOG<n>`), telemetry paced in game time, with the settings that differ from the defaults. Tunables live only in `config.py`; `--set FIELD=VALUE` overrides one for a flight. Six reference saves (`quicksave`, `qs_hot`, `qs_cold`, `qs_steep`, `qs_north` for the 17 t booster; `qs_heavy` for the 99 t five-engine booster) give the separation states; `tools/savegen.py` writes more, and `tests/fakeksp` runs the closed loop offline for screening. The farm flies the booster at 1× (it has no time-scale governor).

State of the code on 2026-10-16 (defaults e7e5aa99). Tunables: `boosterland/config.py`.